Lab No: 04

Lab Title: **The switch and break statement in C++**

- **Learning objectives:**

The following topics will be covered in this lab session

- What is break statement
- Flow chart of break statement
- continue statement with for loop
- continue statement with while loop
- What is continue statement
- Flow chart of continue statement
- continue statement with for loop
- continue statement with while loop
- Comparison of Break and Continue Statement

- Theory

Switch Statement

The switch statement allows a variable to be tested for equality against a list of values called cases.

Syntax

```
switch(expression)
{
   case constant1:
      // statements
      break;

   case constant2:
      // statements
      break;

   default:
      // statements
}
```

- *Each case represents a possible value of the expression.*
- *The break statement stops execution of the switch block.*
- *The default case runs if no case matches.*

What is break statement?

The **continue** statement is used inside loops. Whenever a continue statement is encountered inside a loop, control directly jumps to the beginning of the loop for next iteration, skipping the execution of statements inside loop's body for the current iteration.

The **break statement** is used in following two scenarios:

- Use break statement to come out of the loop instantly. Whenever a break statement is encountered inside a loop, the control directly comes out of loop terminating it. It is used along with if statement, whenever used inside loop (see the example below) so that it occurs only for a particular condition.
- It is used in switch case control structure after the case blocks. Generally, all cases in switch case are followed by a break statement to avoid the subsequent cases (see the example below) execution. Whenever it is encountered in switch-case block, the control comes out of the switch-case body.

Example: break statement in Switch Case.

```
#include <iostream>
using namespace std;
int main(){
  int num=2;
  switch (num) {
    case 1: cout<<"Case 1 "<<endl;
    break;
    case 2: cout<<"Case 2 "<<endl;
    break;
    case 3: cout<<"Case 3 "<<endl;
    break;
    default: cout<<"Default "<<endl;
  }
  cout<<"Hey, I'm out of the switch case";
  return 0;
}
```

Output:

```
Case 2
Hey, I'm out of the switch case
```

- In this example, we have break statement after each Case block, this is because if we don't have it then the subsequent case block would also execute.

  - The output of the same program without break would be:

Example: Use of break statement in a while loop.

```
#include <iostream>
using namespace std;
int main () {
  int var;
  for (var =200; var>=10; var --) {
    cout<<"var: "<<var<<endl;
    if (var==197) {
      break;
    }
  }
  cout<<"Hey, I'm out of the loop";
  return 0;
}
```

What is continue statement?

The **continue** statement is used inside loops. Whenever a continue statement is encountered inside a loop, control directly jumps to the beginning of the loop for next iteration, skipping the execution of statements inside loop's body for the current iteration.

- **Example of continue statement with for loop in C++**

```
#include <iostream>
using namespace std;
int main ()
{
        for (int i = 1; i <= 10; i++)
          {
                if (i == 5)
                {
                continue;
                }
                cout << i << endl;
          }
 return 0;
}
```

- This C++ program iterates 10 times using a for loop.
- The continue statement is encountered when i=5. So, it skips the remainder of the loop's body for that iteration.
- As a result, it only prints the numbers 1-4 and 6-10, skipping 5.

Output:

- **Example of continue statement with for loop in C++**

```
#include <iostream>
using namespace std;

int main ()
{
 int i = 0;
 while (i < 10)
        {
        i++;
        if (i == 5)
                {
                continue; //the iteration is skipped
                }
                else
                {
                // prints the value of i
                cout << i << " ";
                }
        }
 return 0;
}
```

- The above program prints the numbers from 1 to 10 using a while loop in C++
- When i becomes 5, the continue statement is encountered. Therefore, the iteration at i=5 is skipped
- The iteration again starts with i=6 and prints the value from 1 to 10 skipping 5.

Output:

```
0 1 2 4 5
0 1 2 4 5
0 1 2 4 5
```

Comparison of Break and Continue Statements in C++		
Parameters	**break statement in C ++**	**continue statement in C++**
Purpose	Exits the current loop	Skips the current iteration
Applicability	Used within loops (**for, while, do-while**)	Used within loops (**for, while, do-while**)
Effect	Terminates the loop prematurely and continues with the next statement after the loop	Immediately moves to the next iteration of the loop, skipping the remaining code in the current iteration
Loop Control	Affects the entire loop	Affects only the current iteration of the loop
Typical Use Cases	When a specific condition is met, and you want to exit the loop	When you want to skip the current iteration of the loop based on a certain condition and continue with the next iteration

Exercise:

1. Demonstrate continue statement with do...while loop in C++

Input:

```
#include <iostream>
using namespace std;
int main () {
   int num = 1;
   do {
      if (num % 2 == 0) {
         num++;
         continue;
      }
      cout << num << " ";
      num++;
   } while (num <= 10);
   cout << endl;
   return 0;
}
```

Output:

```
1 3 5 7 9
```

2. Demonstrate break statement with while loop in C++

❖ **Input:**

```
#include <iostream>
using namespace std;
int main() {
    int num = 1;
    while (num <= 10) {
        if (num == 6) {
            break; }
        cout << num << " ";
        num++;
    }
    cout << endl;
 return 0;
}
```

❖ **Output:**

```
1 2 3 4 5
```

Lab Tasks

- **Task 1:**
- **1. Write a C++ program that takes an input of a day's number and uses a switch state ment to output the corresponding day's name. Here's how the program will proceed:**
- The program prompts the user to enter the day's number. The user enters a number between 1 and 7, where 1 is Monday and 7 is Sunday.
- The program uses a **switch** statement to check which day was entered.
- Depending on the entered number, the corresponding day's name is outputted using the **cout** statement.
- The **break** statement is used to end each case statement and prevent the program from executing subsequent case statements.
- Finally, the program returns 0 to indicate successful execution
- **Input:**

```
    #include<iostream>
      using namespace std;
       int main()
{
    char A;

    cout<<"enter A: ";
    cin>>A;
    switch (A)
    {
    case '1':
       cout<<"Monday"; /* code */
        break;
        case '2':
        cout<<"Tuesday"; /* code */
         break; case '3':
         cout<<"Wednesday"; /* code */
          break; case '4':
          cout<<"Thursday"; /* code */
           break; case '5':
           cout<<"Friday"; /* code */
            break; case '6':
            cout<<"Saturday"; /* code */
             break; case '7':
             cout<<"Sunday"; /* code */
              break;
    default:
    cout<<"non of these ";
        break;
    }
    return 0;

}
```

Output:

Task 2: Write a C++ program using switch that takes a character grade (A, B, C, D, F) from the user and

prints:

A → Excellent

B → Very Good

C → Good

D → Pass

F → Fail

If any other character is entered, display "Invalid Grade"

```
#include <iostream>
using namespace std;

int main()
{
   char grade;

   cout << "Enter grade: ";
   cin >> grade;

   switch(grade)
   {
      case 'A':
         cout << "Excellent";
         break;

      case 'B':
         cout << "Very Good";
         break;

      case 'C':
         cout << "Good";
         break;

      case 'D':
         cout << "Pass";
         break;

      case 'F':
         cout << "Fail";
         break;

      default:
         cout << "Invalid Grade";
   }

   return 0;
}
```

Task 3:

Write a C++ program using switch that displays the following menu:

1 → Convert Kilometers to Meters 2 → Convert Meters to Centimeters 3 → Convert Kilograms to Grams 4 → Convert Celsius to Fahrenheit

The program should:

Take user choice Ask for input value

Perform the selected conversion Display the result

Show "Invalid choice" if input is wrong

```
#include <iostream>
using namespace std;

int main()
{
    int choice;
    float value;

    cout << "Conversion Menu" <<
endl;
    cout << "1. Kilometers to Meters"
<< endl;
    cout << "2. Meters to Centimeters"
<< endl;
    cout << "3. Kilograms to Grams"
<< endl;
    cout << "4. Celsius to Fahrenheit"
<< endl;

    cout << "Enter your choice: ";
    cin >> choice;

    cout << "Enter value: ";
    cin >> value;

    switch(choice)
    {
        case 1:
```

```
        cout << "Meters = " << value *
1000;
        break;

      case 2:
        cout << "Centimeters = " <<
value * 100;
        break;

      case 3:
        cout << "Grams = " << value *
1000;
        break;

      case 4:
        cout << "Fahrenheit = " <<
(value * 9/5) + 32;
        break;

      default:
        cout << "Invalid choice";
   }

   return 0;
}
```

Conclusion

In this lab session, we learned about the **switch statement, break statement, and continue statement in C++**. The switch statement helped us handle multiple conditions more efficiently compared to multiple if-else statements. We also created simple programs including a **day selector, grade evaluator, and a menu-driven conversion program**, which improved our understanding of decision-making statements in programming.